

Final Relational Model (Post Merge and Normalization)

Ayesha Qadir, Kriza Cyrene del Moro, Jatinder Sahota
10/12/25

constructor (constructorID, constructorRef, name, nationality)

season (year)

race (raceID, round, date, grandPrixOrSprint, FK circuitID REF circuit, FK year REF season)

circuit (circuitID, name, location within country, location, latitude, longitude, altitude)

driver (driverID, driverRef, firstName, lastName, dateOfBirth, nationality)

lap (lapNumber, lapTime, position, FK raceID REF race, FK driverID REF driver)

partOf (FK driverID REF driver, FK constructorID REF constructor, FK year REF season)

constructorRace (FK raceID REF race, FK constructorID REF constructor, constructorPoints)

driverRace (FK raceID REF race, FK driverID REF driver, driverPoints, gridPosition, finalPosition, rank)

driverChampionship (FK driverID REF driver, FK year REF season, winner)

constructorChampionship (FK constructorID REF constructor, FK year REF season, winner)

Steps for merging and normalizing are on the next page.

The only things that needed to be merged were the “happens on” and “makes up” relationships into the “race” table. Those were both 1-to-many relationships with “race” being on the many side, and since they both concerned a specific instance of the race, we collapse the two tables and insert the primary keys as foreign keys into “race.” Constructor, season, and race were all in 3NF already, so no normalizing was done there.

Circuit had a compound attribute in location, which was broken up into two distinct attributes. In our database, computing the country and location within country through latitude, longitude, and altitude does not seem feasible, and so we do not consider those three attributes to determine location. With this assumption, breaking up the compound attribute in circuit allowed it to normalize into 3NF.

Driver just needed the name broken up into first and last name to remove any compound attributes, then all requirements for 3NF are satisfied.

The “recorded-in” relationship concerned the laps that a single driver executed in a single race, and so the relationship is collapsed and its attributes are moved into Lap as foreign keys (aside from lap number). Requirements for 3NF are satisfied.

For the “part-of” relationship, since drivers can be in multiple constructors overtime, there would be mapping problems since a single driver can map to multiple constructors without additional information, so we introduced “year” as a foreign key to help identify which exact constructor the driver is apart of at a given time. Since “part-of” is many-to-many, it gains its own table, where as mentioned, we provide it a year to reference Season. All requirements for 3NF are satisfied.

For the “races in” relationship, it is many-to-many so we require a new table, but it also has partial key dependency since “driver points” would not need “constructor ID” to see the points they earned in that instance. We break the relationship up into two tables, one pertaining to the race participation concerning individual drivers, and one concerning constructors. Each relation will have the relevant attributes from “races in” (i.e., ConstructorRace will not have driver points or grid position), and each relation has no partial key dependency. There is no transitive dependency, and so 3NF is satisfied.

The “participates in” relationships for constructor/driver and the season just need to become tables, and so they are converted into “driverChampionship” and “constructorChampionship.” The tables satisfied 3NF.